A smaller proof presentation of intuitionistic primitive recursive arithmetic

Deterministic introduction with auxiliary induction

Prepared for Vladimir Reshetnikov
with ChatGPT

11 November 2026

Abstract

The current ConciseIPRA project has two separately established reductions: transport can exclude a literal-zero source, and a guarded public language can use deterministic introduction. We prove that they can coexist when unguarded atomic progress formulas remain available as auxiliary intermediate formulas. Auxiliary equation induction supplies numerical cases before equality is repaired. The numerical zero-source argument then applies, and full transport is recovered for public guarded formulas. The resulting public consequence relation is exactly the existing guarded one; its ordinary realization is therefore bi-interpretable and equiconsistent with full first-order iPRA, with both numerical composites the identity.

Under the repository's canonical-statement checking contract, the measured coordinate envelope decreases from 259 significant tokens, 283 AST nodes, and 526 normalized bytes to 258, 281, and 522. Different programs attain these coordinates. The direct mathematical composition gives 525 bytes; the remaining three bytes come from an optional parity spelling. A separate checker including public syntax validation has 1,205 bytes, compared with 1,208 for the same guarded validator and 1,109 for the older unrestricted native control. Thus no global strict-checker or absolute-minimality claim is made. The new metatheorem has a written proof and executable examples, not a new Lean or Rocq formalization.

Contents

1 Result, comparison target, and the meaning of smaller **3**

2 A deterministic calculus with auxiliary formulas **4**

3 What the executable checker implements **10**

4 Verification, provenance, and limits **12**

5 Further research questions **13**

Inherited mathematics supplied for offline review **14**

A Complementary introductions with one local assumption **14**

B Arithmetic strength and both ordinary interpretation composites **19**

C Internal scalar predecessor bootstrap **21**

D From literal-equation induction to full intuitionistic PRA **25**

E Pairing, primitive recursion, and internal numerical composites **30**

1 Result, comparison target, and the meaning of smaller

The comparison is pinned to ProveIt commit

`1517d9d79ce33f631e0f82960c849427cbba23bf`.

The target directory is `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. Its README records 28 research packages, the native scalar core, and the Lean bridges to full first-order iPRA. The inspected frontier lists `source_dispatch` at 259 tokens, 283 AST nodes, and 526 normalized bytes, each attained by its corresponding profile. The `guarded_core` package separately removes the introduction-mode choice. This report combines those mathematical interfaces under an explicit public/auxiliary distinction [1, 2, 3].

The change preserves the existing scalar arithmetic foundation. It does not introduce a universal primitive-recursive evaluator, a restriction to standard numerals, a quotient equality, or a stronger induction schema. The substantive new statement is the compatibility theorem in Section 2. The syntactic guard and the numerical zero-source argument are inherited and are credited as such.

1.1 Measured programs

Every row charges the entire standalone module, including input, opening and substitution, scopes, contexts, arithmetic supplies, logical checking, and the final verdict. CPython and its standard JSON parser are common runtime infrastructure, following the existing convention. Normalize with `ast.unparse(ast.parse(source))` and one final newline, under CPython 3.12.14. Count every AST node; significant tokens exclude comments, whitespace, indentation, and stream markers. Gzip uses level 9 and `mtime=0`, with its wrapper included. Gzip is informational.

Complete program	Tokens	AST nodes	Bytes	Gzip-9
`checker.py`	265	303	525	331
`checker_ast.py`	262	281	606	348
`checker_bytes.py`	265	303	522	329
`checker_strict.py`	491	563	1205	572
`checker_tokens.py`	258	287	571	339

For comparison, the previous source-dispatch byte, AST, and token profiles respectively measure

	tokens	AST	bytes	gzip-9
byte profile	266	305	526	330
AST profile	263	283	607	348
token profile	259	289	572	339.

The new compact envelope is therefore

258 tokens, 281 AST nodes, 522 bytes

.

No single program has all three minima. The readable annotated checker is also supplied and is measured in the machine-readable ledger; it is not used to claim a minimum. All exact source files and their measurements are in the archive.

The direct 525-byte version has gzip size 331, one byte larger than the previous byte profile's gzip size 330. The optional 522-byte profile has gzip size 329. The comparison does not optimize gzip or conceal this distinction.

1.2 Public syntax validation has a separate cost

The compact programs assume that the proposed end statement belongs to the specified public grammar. They may also accept proofs of auxiliary raw goals. This is the same kind of contract used by the existing guarded package; it is not a claim that arbitrary JSON is validated as a public statement. The strict program includes that validation in its source size.

Strict checker	Tokens	AST nodes	Bytes	Gzip-9
Existing guarded grammar	491	568	1208	574
New, identical guarded validator	491	563	1205	572
Older unrestricted native control	450	521	1109	538

The first two rows are a matched comparison with identical validation functions. The third uses a less restrictive public grammar and is smaller than the new strict checker. Consequently the compact-coordinate result does not establish an overall record when validation of every public goal is charged across all grammars. The observed minima are upper bounds on an unknown optimum, not proved lower bounds.

2 A deterministic calculus with auxiliary formulas

The construction combines two restrictions that have different roles. The source of an equality elimination must not be the literal term 0, and the root of a paired matrix determines its introduction rule. There is no independent case/progress choice. The interaction is resolved by allowing auxiliary formulas in proofs. Only the public formulas below are assigned the ordinary conditional reading in the bi-interpretation theorem. In particular, we do not assert a round-trip theorem for every auxiliary formula.

2.1 Terms, public formulas, and proof formulas

There is one numerical sort. The terms are

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h),$$

where u is bound only in h. Substitution is capture-free. Write

$$1 := S0, \qquad 2 := S1, \qquad C(v, a, b) := I(v, a; u.b),$$

with u fresh in the last abbreviation. For an equation $E \equiv (s = t)$, put

$$G_v E := \big(C(v, 0, s) = C(v, 0, t)\big).$$

The public language $\mathcal{G}$ is the guarded language of the inherited package: every equation is public; $\mathsf{D}_{x,y}B$ is public if B is a compound public formula; and an atomic matrix immediately under $\mathsf{D}_{x,y}$ must have the literal shape

$$B(x, y) \equiv I(y, 0; u.h) = I(y, 0; u.k). \tag{G}$$

The bodies h, k may contain x, y, the state variable, and fixed parameters. This is a syntactic condition. Every such matrix has a provable zero instance. The designated contradiction is $0 = 1$.

The auxiliary proof language $\mathcal{U}^-$ is larger:

$$A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A.$$

Its paired matrices need not satisfy (G). Contexts are finite lists of reusable auxiliary assumptions. Public theoremhood means that a public conclusion has a derivation in this auxiliary calculus; it does not require every intermediate formula to be public. The same definition applies to consequence from public assumptions.

2.2 All primitive axioms and rules

The only nonlogical axiom schemas, available under every context, are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0,IS}$$

The hypothesis rule is $\Gamma \vdash A$ whenever $A \in \Gamma$. The remaining rules are

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \quad (s \text{ is not literally } 0), \tag{Tr$^-$}$$

$$\frac{\Gamma \vdash \mathsf{Q}_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}, \tag{QE}$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x, y) \qquad \Gamma, \mathsf{Q}_y B(a, y) \vdash A}{\Gamma \vdash A}, \tag{DE}$$

and one root-determined introduction schema

$$\frac{\Gamma, B(t, \eta_B(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}, \qquad \eta_B(z) = \begin{cases} z, & B \text{ is a literal equation,} \\ 0, & B \text{ has root } \mathsf{Q} \text{ or } \mathsf{D}. \end{cases} \tag{DI$_{\text{root}}$}$$

Here F is an auxiliary formula template. In DE, a is fresh for the retained context, the conclusion, and the fixed parameters. In DI, z is fresh for the retained context, the witness term t, and the fixed parameters. The displayed local assumption is discharged. Bound names are always chosen to avoid capture.

There is no primitive equality reflexivity or successor axiom, no separate induction schema, no unrestricted atomic case introduction, and no primitive introduction rule for Q. Weakening, renaming, contraction, and replacement of a retained assumption by its proof are admissible by the usual induction on a finite derivation. No unrestricted numerical-substitution theorem for auxiliary derivations is needed below: a substitution could turn an allowed source variable into the literal 0, so a direct constructor-wise substitution argument would be invalid.

2.3 Induction and cases are available before equality is repaired

Lemma 2.1 (Auxiliary equation induction)**.** *For every literal equation $E(y)$, the rule*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}$$

is admissible, with the usual fresh z. The retained context may contain arbitrary auxiliary formulas.

Proof. Apply atomic DI to $\mathsf{D}_{x,y}E(y)$, with x unused and witness 0. Its premise is precisely the supplied step. DE exports $\mathsf{Q}_y E(y)$; the fresh witness is absent from this formula. QE and the supplied base give $E(t)$. This derivation uses no transport at all. Its paired intermediate formula is generally auxiliary rather than public. □

Lemma 2.2 (Numerical cases)**.** *From $\Gamma \vdash A(0)$ and $\Gamma \vdash A(Sz)$, with z fresh, one may infer $\Gamma \vdash A(t)$ for every auxiliary $A(y)$.*

Proof. For an equation, use Lemma 2.1 and ignore its local induction assumption. For a compound formula, case DI with an unused witness gives $\mathsf{D}_{x,y}A(y)$; DE and QE use the given base to obtain $A(t)$. □

This is the decisive bootstrap. We do not first try to pad the hypothesis $0 = t$ into a public compound formula. Auxiliary equation induction is already available, and supplies the atomic cases needed by the literal-zero-source argument.

These are the compatibility steps needed to combine the two previously separate reductions. The guarded public grammar is inherited from [3]; the source restriction is inherited from [2].

2.4 Recovering equality from the restricted source rule

The following proof reconstructs the zero-source argument in the pinned `source_dispatch` package and `NativeZeroSource.lean`. Its only non-equational ingredients are equation induction and numerical cases, now supplied by Lemmas 2.1 and 2.2. The formulas in the calculations below are equations. The new proof reuses this numerical argument only; the final unrestricted auxiliary-formula transport argument of the source package is not used [2, 9].

Lemma 2.3 (Safe equality calculations)**.** *Reflexivity is derivable for every term. Symmetry is derivable for $p = q$ when p is not literally 0, and transitivity is derivable through any middle term that is not literally 0. Congruence and replacement on either side of an equation are available along an equation with such a source. If p, q are not literally 0, then*

$$\Gamma \vdash p = 0, \quad \Gamma \vdash q = 0 \quad \Longrightarrow \quad \Gamma \vdash p = q. \tag{Zero join}$$

Proof. Put $r = I(0, t; u.0)$. The axiom $r = t$, used as both premises of Tr^- with template $v = t$, gives $t = t$. The usual equality templates then give symmetry at an allowed source, transitivity through an allowed middle term, and congruence at an allowed source.

For the last assertion, let $W(v) = I(0, v; u.0)$ and $w = W(0)$. From $p = 0$ and $W(p) = W(p)$, replace p only on the left to obtain $w = W(p)$. Symmetry at w, followed by the computation $W(p) = p$ on the left, gives $p = w$. Similarly $q = w$. Symmetry at q and transitivity through w give $p = q$. All these sources and the middle term w are syntactically nonzero; no numerical inequality is asserted. □

Lemma 2.4 (Zero-source symmetry)**.** *For every term t and every auxiliary context,*

$$\Gamma \vdash 0 = t \quad \Longrightarrow \quad \Gamma \vdash t = 0.$$

Proof. The case where t is literally 0 is reflexivity. Otherwise t is an allowed source. Write

$$A(n) = I(n, t; u.Su).$$

The zero computation and safe symmetry give $t = A(0)$, which may replace the right side of the assumed $0 = t$; thus $0 = A(0)$. Under $z = A(z)$, congruence at the variable source z gives $Sz = S(A(z))$, and the successor computation gives $Sz = A(Sz)$. Auxiliary equation induction therefore gives $1 = A(1)$ and hence

$$1 = St. \tag{1}$$

For completeness, the small test used to remove this successor is

$$\begin{aligned} Z(v) &= C(v, 1, 0), & K(v) &= C(v, 0, 1), & H(v) &= C(v, 1, 2), \\ N(n) &= I(n, 0; u.Z(u)), & R(n) &= I(n, 0; u.H(u)), & L(n) &= N(R(n)). \end{aligned}$$

Numerical cases and computation give $H(v) = S(K(v))$. Consequently $R(1) = 1$ and $R(SSv) = 2$. Computation gives $N(1) = 1$ and $N(2) = 0$, so $L(1) = 1$ and $L(SSv) = 0$. A second numerical case argument yields

$$L(Sv) = Z(v). \tag{2}$$

At the zero branch both sides reduce to 1; at the successor branch both sides reduce to 0, and (Zero join) joins them. All reductions use iterator sources. Similarly numerical cases prove the reversed filter identity

$$C(Z(v), v, 0) = v. \tag{3}$$

Its zero branch reduces via $Z(0) = 1$, and its successor branch via $Z(Sw) = 0$. Again every major used for reduction has a nonzero syntactic source.

Applying L to (1), using $L(1) = 1$ and (2), and using safe symmetry and transitivity, gives $Z(t) = 1$. Hence the left side $q = C(Z(t), t, 0)$ of (3) is also provably 0. The filter gives $q = t$. Symmetry at the iterator source q, followed by transitivity through q, yields $t = 0$.

Every identity just used is a schema constructed at the displayed terms. The argument does not assume that arbitrary restricted derivations may be numerically substituted. □

Corollary 2.5 (Unrestricted equational transport)**.** *Symmetry, transitivity, term congruence, and transport in every equation template are admissible without a source restriction.*

Proof. Symmetry follows from Lemmas 2.3 and 2.4. For transitivity through literal 0, the cases of a literal-zero endpoint are immediate; otherwise reverse the second equation and use zero join. For congruence along $0 = t$ with t not literally 0, reverse to $t = 0$, use safe congruence, then use unrestricted symmetry. If $F(v)$ is $a(v) = b(v)$, term congruence, symmetry, and transitivity transform $F(s)$ to $F(t)$. □

Lemma 2.6 (Transport in public formulas)**.** *The unrestricted transport rule is admissible whenever its template F is public. Thus numerical equality has its full ordinary use throughout the public language.*

Proof. Induct on the public formula template. The equation case is the preceding corollary. Suppose first that $F(v) = \mathsf{D}_{x,y}B(v, x, y)$ has compound matrix. Open $F(s)$ at a fresh witness a, obtaining the clause $\mathsf{Q}_y B(s, a, y)$. Introduce $F(t)$ at witness a by compound DI. Its base assumption $B(t, a, 0)$ transfers, using the induction hypothesis and $t = s$, to $B(s, a, 0)$. QE gives $B(s, a, Sz)$; the induction hypothesis and $s = t$ transfer this to $B(t, a, Sz)$.

If B is atomic, its public guard makes $B(s, a, 0)$ a theorem. The opened source clause therefore gives $B(s, a, Sz)$ outright. Unrestricted equational transport gives $B(t, a, Sz)$. Atomic DI introduces the target, ignoring its local induction assumption. Close DE in both cases. Eigenvariables are chosen fresh for all fixed parameters. □

This lemma is intentionally about public templates. We do not need, and do not assert, unrestricted zero-source transport in every auxiliary formula containing an unguarded atomic paired matrix.

2.5 Soundness of every auxiliary derivation

Give auxiliary formulas the following ordinary first-order readings over the scalar terms:

$$(s = t)^\dagger = (s = t),$$
$$(\mathsf{Q}_y B)^\dagger = B^\dagger(0) \to \forall y\, B^\dagger(y),$$
$$(\mathsf{D}_{x,y} B)^\dagger = \begin{cases} \exists x\, \forall y\, (B(x,y) \to B(x,Sy)), & B \text{ is an equation,} \\ \exists x\, (B^\dagger(x,0) \to \forall y\, B^\dagger(x,y)), & B \text{ is compound.} \end{cases}$$

Numeric substitution preserves the root category of a formula, so this reading commutes with capture-free numeric substitution. The reading of Q is always conditional, including at an equation.

Theorem 2.7 (Auxiliary soundness)**.** *If $\Gamma \vdash_{\mathcal{U}^-} A$, then iPRA proves $A^\dagger$ from $\Gamma^\dagger$, after the inherited primitive-recursive realization of scalar terms.*

Proof. Induct on the derivation. Hypothesis, computation, transport, and QE have their ordinary sound interpretations. Atomic DI directly proves the displayed progressive reading. Compound DI uses numerical coverage to extend its zero-to-successor premise to a zero-to-all conclusion. For DE at an atomic matrix, ordinary equation induction converts $\forall y(E(y) \to E(Sy))$ into $E(0) \to \forall y E(y)$, which is exactly the exported Q assumption. This uses only quantifier-free induction, with the existential witness and other parameters fixed. DE at a compound matrix is ordinary existential elimination. All side assumptions and eigenvariable conditions are preserved. □

Let A° denote the *uniform ordinary reading* on public formulas: equality is actual numerical equality and every public $\mathsf{D}_{x,y}B$ means $\exists x(B^\circ(x,0) \to \forall y B^\circ(x,y))$.

Lemma 2.8 (Agreement on public formulas)**.** *For every public A, iPRA proves $A^\dagger \leftrightarrow A^\circ$.*

Proof. At a guarded atomic matrix $E(x,y)$, the zero instance $E(x,0)$ is a theorem. Progress implies all instances by equation induction, while conditional closure implies all instances by that true base and hence implies progress. These two readings are therefore equivalent, with x fixed, and remain equivalent under the existential quantifier. At compound matrices use structural induction. Equations are unchanged. □

2.6 Completeness and both interpretation composites

We record the inherited guarded translation explicitly, because it identifies the exact scope of the result. Let $\mathcal{C}$ be the existing native scalar calculus. Define

$$L(E) = \mathsf{D}_{u,v} G_v E, \qquad P(A) = \begin{cases} L(A), & A \text{ an equation,} \\ A, & A \text{ compound.} \end{cases}$$

The new variables in L are fresh. Translate old Eq/D formulas by

$$T(s = t) = (s = t), \qquad T(\mathsf{D}_{x,y} B) = \mathsf{D}_{x,y} P(TB).$$

If the old presentation also treats Q as an object constructor, put $T(\mathsf{Q}_y B) = \mathsf{D}_{x,y} P(TB)$ with x unused. Let J be the literal inclusion of public formulas into the old language.

Lemma 2.9 (Guarded translation and its composites)**.** *The translation T preserves derivations of the old calculus in the new public consequence relation. Uniformly under additional contexts,*

$$A \dashv\vdash_{\mathcal{C}} JT(A), \qquad B \dashv\vdash_{\mathcal{U}^-} TJ(B) \quad (B \text{ public}).$$

Proof. After equality repair, $L(E) \dashv\vdash E$: from E, atomic DI proves $L(E)$ by ignoring its local assumption; conversely DE, the theorem G_0E, and QE at 1 recover E. Therefore $P(A)$ is always compound and interderivable with A.

Old case introduction transfers to compound DI after padding the base and the successor conclusion. Old equation progress transfers by auxiliary equation induction under the padded base assumption, followed by compound DI. The old direct Q progress rule is covered by Lemma 2.1 and the unused-witness translation of Q. Elimination transfers by replacing each use of its opened clause with padding of the antecedent, use of the translated clause, and unpadding of the consequent. Transport transfers by Lemma 2.6. Computation and assumptions are unchanged. This is an effective transformation of finite proofs.

The old calculus has binder congruence and the same padding equivalence, so structural induction gives its round trip. In the new calculus, uniform equivalence of two compound matrices lifts under D by DE and compound DI. At a guarded atomic matrix B, the remaining step is

$$\mathsf{D}_{x,y}B \dashv\vdash \mathsf{D}_{x,y}L(B).$$

The forward direction opens the source clause, unpads the target base, uses the clause, and pads the successor. For the reverse direction, the true guarded base proves $L(B(a,0))$; the opened clause then gives $L(B(a,Sz))$ and hence $B(a,Sz)$. Atomic DI introduces $\mathsf{D}B$, ignoring its local assumption. These are all the cases in the new round-trip induction. An unused existential witness introduces or eliminates the old Q interpretation, using the witness 0. □

Theorem 2.10 (Exact public consequence relation)**.** *For public Γ, A,*

$$\Gamma \vdash_{\mathcal{U}^-} A \iff \Gamma \vdash_{\mathcal{C}} A \iff \Gamma^\circ \vdash_{\mathsf{iPRA}} A^\circ.$$

Proof. Auxiliary soundness and public agreement give the rightward implication from the new calculus to iPRA. The pinned native–iPRA equivalence gives the old native derivation. Conversely, translate an old derivation by T and use both public round-trip equivalences, including the uniform equivalences for the retained assumptions, to recover the original public sequent. The second equivalence is the inherited native–iPRA bridge. □

Corollary 2.11 (Bi-interpretability and equiconsistency)**.** *The ordinary realization of this public theory is bi-interpretable and equiconsistent with full first-order iPRA. Both numerical domains are the whole domain; equality is actual equality; and both numerical composites have the identity graph $x = y$.*

Proof. Compose T, J and their formula composites with the pinned bi-interpretation of the native calculus and iPRA. No term or numerical sort is changed by T or J, so the numerical composites remain the identity. The distinguished contradiction $0 = 1$ is unchanged and has the ordinary reading equivalent to falsity in iPRA. Exact public consequence therefore gives equiconsistency as well. □

2.7 Why the auxiliary distinction is essential

Proposition 2.12 (The canonical-only composition would fail)**.** *If paired matrices throughout derivations were required to satisfy the public grammar, with* Q *resources only the corresponding opened clauses, the simultaneous restrictions would not even prove* $1 = 0$ *from* $0 = 1$*.*

Proof. Interpret terms in the standard natural numbers. At the root of a formula, declare every literal equation $0 = t$ true, and interpret every other equation by actual equality. Interpret public compounds by their ordinary conditional reading. Every atomic leaf under a public paired binder has an iterator on the left; consequently those leaves always receive actual equality, and their readings commute with substitution.

Computations, guarded atomic progress, compound case introduction, and the associated clause elimination are sound for this interpretation. Restricted transport is sound too. Its major $s = t$ has a syntactically nonzero left side, so it asserts actual equality. If an equation template instantiated at s has literal 0 on the left, that left template was constant 0: it could not have been the hole, since s is not literally 0. The target is then also true by convention. Otherwise transport preserves actual equality, unless its target has literal-zero left side and is true by convention anyway. Transport inside a public compound is ordinary extensional transport.

The assumption $0 = 1$ is true in this interpretation, while $1 = 0$ is false. This proves the stated nonderivability. The auxiliary induction used in Lemma 2.4 is exactly the extra proof resource that invalidates this counterinterpretation. □

Remark 2.13 (Exact checking contract)*.* The compact checker is a checker of auxiliary derivations with a *canonical public-goal contract*. Its soundness and completeness claim is Theorem 2.10, for a goal in $\mathcal{G}$. A raw auxiliary goal need not be public, even when the program accepts its proof. The separately supplied strict checker includes and charges an explicit public-goal validator. A claim about the compact kernel alone must not silently include the validator, and a claim about the whole auxiliary syntax must not be substituted for the public bi-interpretation proved here.

3 What the executable checker implements

3.1 The source selects elimination; the matrix selects introduction

The two eliminations in the mathematical inventory can be written as one source-selected pattern. For a template $F(v)$, define

$$M(s,t;F) = \begin{cases} \mathsf{Q}_v F(v), & s \equiv 0, \\ s = t, & s \not\equiv 0. \end{cases} \qquad \frac{\Gamma \vdash M(s,t;F) \quad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}.$$

Here $s \equiv 0$ is a syntactic condition. A nonliteral-zero term may of course denote the number zero. The case/progress introduction choice is also absent: equation matrices receive their fresh induction instance; compound matrices receive their zero instance. These facts determine the two branches which the certificate formerly had to select separately.

The direct complete implementation is:

```
import json

def c(a, g=[[]], k=0):
```

```
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 == [] else a if k <= a < d else d % 2
     and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    a = m == ([s, u] if s else [t] * 3) and i(t, u) == a and i(t, s) or a
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a := i(*a, t, 2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or not len(a) % 2) and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

Relative to the corresponding pinned source-dispatch checker, the only substantive source change is

```
len(a) % 2 == u
```

becoming

```
not len(a) % 2
```

in the local profiles. The global token profile formerly compared with `m`. On meaningful matrix roots, equality has even arity and a binder has odd arity. This substitution is justified by the compatibility theorem, not by assuming that the old and new certificate languages coincide.

The optional byte spelling is

```
~len(a) % 2
```

because, for a nonnegative integer r, $(\sim r) \bmod 2 = 1 - (r \bmod 2)$ in Python's integer operations. The expression is consumed only through its truth value and numerical zero/one behavior. It saves three additional normalized bytes. It is an implementation choice, separate from the mathematical compatibility result.

3.2 Certificate format and binding

Input is a JSON array. The last element is the closed proposed formula; earlier elements are four-field instructions consumed from right to left. Unused prefix data is inert. Success is normal exit with exactly the line `ACCEPT`; exceptions, nonzero exits, and exhausted physical resources are rejection. The mathematical completeness statement abstracts from a fixed machine's finite resources.

The inherited arity representation has the following sorted meanings:

Array length	At a numerical position	At a formula position
0	zero annotation	truth annotation
1	successor	paired D
2	zero annotation	equation
3	scalar iterator	auxiliary Q
≥ 4	zero annotation	truth annotation

Canonical zero is the integer 0. Bound slots have width two; numerical root depth is three. The opening traversal adds $5 - r$ beneath an array of length r. Successor has two dummy zero slots; an iterator has one slot, dummy in its counter and initial value and genuine in its update. Negative

integers allocated by the checker are fresh rigid parameters. The strict validator admits the same canonical odd-slot convention as its predecessor.

For local profiles an instruction is $[m, s, u, t]$. A proved unary major $m = [B]$ exports the auxiliary closure $[B, B, B]$ at a fresh witness. With the neutral major $m = []$, a nonempty u supplies a computation, and a falsy u starts introduction; the matrix root determines its local assumption. A proposed equality or closure elimination must first match its exact major and endpoints, and its major must actually be checked. The token profile uses the inherited global name supply; the other compact profiles use lexical scopes. Producers therefore emit distinct local and global packets. Those formats are not silently interchanged.

Raw aliases have the explicit zero or truth readings inherited from the original encoding. Meaningful ternary formulas use their third component as the `Q` body. These details and the original opening proof are supplied in `reference/inherited_arity_checker.tex`; the guarded extension is in `reference/guarded_implementation.tex`. They are credited reference material rather than a new encoding theorem [5, 4].

3.3 Soundness and completeness of the implementation

For soundness, interpret a finite accepting call tree in iPRA. Opening and freshness are inherited. The surviving equality transport and QE are sound. Atomic introduction is progress; compound introduction is cases. Atomic DE exports a conditional closure justified by equation induction in iPRA. For generated computation supplies, the inherited zero-specialization argument is performed in iPRA, where substitution and cut are available. It does not assume an invalid structural substitution theorem inside the source-restricted auxiliary calculus. Truth aliases remain harmless.

On a public conclusion the mixed interpretation agrees with its ordinary interpretation. Theorem 2.10 therefore returns the required public derivation. This argument remains valid even when intermediate raw formulas are auxiliary.

For completeness, encode a finite auxiliary derivation with fresh integer names. The proof construction in Section 2 uses the deterministic introduction modes and only permitted primitive sources. The inherited nonempty zero

$$c_0 = I(2, 1; u.0), \qquad c_0 = 0,$$

repairs computation packets whose initializer or successor predecessor would otherwise be literal zero. Replacing those argument occurrences by c_0 gives a direct packet, followed by transport at source c_0 back to the requested computation equation. This never requires transport from literal zero and does not change the public guard shape of the finished formula [2, 5].

The larger proof producers are untrusted: no checker imports them. The strict checker adds only the public-goal validator to the same kernel. Its validation functions are AST-identical to the pinned guarded control.

4 Verification, provenance, and limits

The main reproduction script checks 33 pinned reference files against their Git blob hashes, regenerates 26 certificates for 13 examples, and executes 242 command-line checks. All passed on CPython 3.12.14, including normal execution and execution with `-O`. Twelve example conclusions are canonical public formulas; the remaining auxiliary `Q` conclusion is accepted by the compact profiles and correctly

rejected by the strict public profile. Independent review also checked seven public-grammar shape controls.

The positive examples include both computation equations, reflexivity, guarded introduction and elimination, compound implication, a guarded induction with a used induction hypothesis, derived zero-source equation transport, and canonical encodings of $0 = t \to t = 0$. The general symmetry fixture keeps an arbitrary successor parameter fixed during the auxiliary induction. Its expanded named proof contains twelve unguarded atomic progress introductions, no atomic case introduction, and no forbidden zero-source transport. This directly exercises the part of the proof which the canonical-only counterinterpretation shows cannot be omitted.

Negative examples exercise future-witness capture, fractional aliases, context leakage, a false sentinel, a previously documented phase-guard attack, and an unproved contradiction. A deliberately old atomic-case certificate is accepted by the old source checker and rejected by the new one. It has a noncanonical end formula. This control records why equality of the two *raw certificate languages* is not claimed.

```
python3 reproduce.py
python3 code/checker_strict.py < tests/certificates/zero_symmetry_all_numbers_local.json
pdflatex -interaction=nonstopmode -halt-on-error ipra_deterministic_auxiliary.tex
pdflatex -interaction=nonstopmode -halt-on-error ipra_deterministic_auxiliary.tex
```

The new compatibility theorem is a written mathematical proof, reviewed independently in this investigation. Executable examples and AST comparison support the implementation; finite tests do not establish universal logical soundness. The existing `native_iff_pra`, `pra_native_roundTrip`, `native_roundTrip`, `native_inconsistent_iff`, and `biInterp` statements were inspected in the pinned repository. Selected source files are included for inspection. They are not a standalone Lean project, and the Lean development was not rebuilt here [10, 11].

The new result is a modest improvement to the observed compact frontier. It is not an optimality theorem, and it does not establish a new arithmetic basis simpler than the scalar iterator. The proof-language distinction is essential; discarding the auxiliary formulas would invalidate the proposed combination. The explicit validator cost is reported rather than hidden.

5 Further research questions

1. Can a different substitution-stable public grammar retain both interpretation composites with a validator no larger than the unrestricted native validator? This targets the gap between compact and strict measurements.
2. Can auxiliary equation progress be a separate proof judgment with a smaller Lean derivation datatype? The target is exact public consequence equivalence, not round trips for all auxiliary formulas.
3. Can the singleton-test argument for zero-source symmetry be shortened? This could reduce certificates without changing the trusted kernel.
4. Which substitution-stable family of auxiliary equation invariants suffices for equality repair and the arithmetic bridge? It must support $n = I(n, t; u.Su)$ with $0 = t$ fixed in the context.
5. Can the joint public theorem and raw sorted-decoder soundness be formalized by extending the pinned Lean development? This would turn the written compatibility argument into a proof-assistant-checked result.

Inherited mathematics supplied for offline review

The following five sections reproduce the pinned repository's reference mathematics. They are inherited results, not new contributions of this report. Their original terminology, including references to earlier "new" changes, is retained so that the supplied source files remain byte-for-byte attributable. The first two sections establish the native logical interface and ordinary bridge; the last three construct predecessor, equality reflection, and the primitive-recursive compiler. The archive records the exact source paths and Git blob hashes in `reference/PROVENANCE.json`. ProveIt's MIT No Attribution license is also included.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \le 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B \rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a,0)$ that clause supplies $B(a,y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* y B$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

B.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem B.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [6] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows

by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{4}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{5}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{6}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{7}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{8}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (8) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{9}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (8); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{10}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (9), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{11}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{12}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (12). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (13) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (14) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (15)$$

The base is (14). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (16)$$

The first uses the comparator recurrence. The second uses (15) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (17)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (18)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (17) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [7]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{19}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{20}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{21}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{22}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[(E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{23}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (23) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{24}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{25}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{26}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (19). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (25); symmetry and (22) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{27}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{28}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{29}$$

To prove the range assertion, split (21). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (20) also give, for all u, v,

$$C(u,v,v) = v. \tag{30}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{31}$$

If $u = 1 = S0$, (26) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{32}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{33}$$

Using these equations and (23), we now derive

$$0 + y = y, \tag{34}$$
$$(Sx) + y = S(x + y), \tag{35}$$
$$d(Sx, Sy) = d(x, y), \tag{36}$$
$$d(Sa, a) = 1. \tag{37}$$

Each induction below has a single equation as its induction formula.

For (34), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (35), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (35) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{38}$$

Indeed, both sides are $S(y + r)$.

For (36), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (37). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (36) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x,y)),\ x,\ y + d(x,y)) = x. \tag{39}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (34) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (30).

For the step, assume (39) at y and abbreviate

$$r = d(x,y), \qquad r' = Pr = d(x,Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (39) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (38) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (39). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa,b) = 1 \quad \longleftrightarrow \quad a = b. \tag{40}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (39). If $d(Sa,b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (37) give $d(Sa,b) = d(Sa,a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}\big(K(d(Sa,b))\big). \tag{41}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{42}$$

$$e(a,b) = 0 \ \vee\ e(a,b) = 1. \tag{43}$$

In detail, $e(a,b) = 0$ is equivalent to $K(d(Sa,b)) = 0$ by (28), which is equivalent to $d(Sa,b) = 1$ by (31), and then to $a = b$ by (40). The range assertion is (27) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a,b) = 0$ branch, (42) gives $a = b$. In the $e(a,b) = 1$ branch, an assumption $a = b$ would give $e(a,b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a\, \forall b\, \big(a = b \vee (a = b \to \bot)\big) \tag{44}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{45}$$
$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{46}$$
$$b_{A\vee D} = C(b_A, 0, b_D), \tag{47}$$
$$b_{A\to D} = C(b_A, b_D, 0). \tag{48}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{49}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (42)–(43). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (49) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (49), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [8]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a,b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

References

[1] ProveIt contributors. *Concise presentations of intuitionistic primitive recursive arithmetic*, README at commit `1517d9d79ce33f631e0f82960c849427cbba23bf`. https://github.com/VladimirReshetnikov/ProveIt/tree/1517d9d79ce33f631e0f82960c849427cbba23bf/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] ProveIt contributors. *Source dispatch*, especially `Research/source_dispatch/proofs/zero_source_admissibility.tex` and `Research/source_dispatch/code/`, at the pinned commit. https://github.com/VladimirReshetnikov/ProveIt/tree/1517d9d79ce33f631e0f82960c849427cbba23bf/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Research/source_dispatch.

[3] ProveIt contributors. *A guarded core with a deterministic introduction rule*, `Research/guarded_core/sections/guarded_core.tex`, pinned commit. https://github.com/VladimirReshetnikov/ProveIt/blob/1517d9d79ce33f631e0f82960c849427cbba23bf/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Research/guarded_core/sections/guarded_core.tex.

[4] ProveIt contributors. *Executable checker and its exact theorem*, `Research/guarded_core/sections/implementation.tex`, pinned commit. Supplied as `reference/guarded_implementation.tex`.

[5] ProveIt contributors. *A uniform arity encoding and its checking theorem*, `Research/certified_zero/reference/inherited_arity_checker.tex`, pinned commit. Supplied unchanged in `reference/`.

[6] ProveIt contributors. *Internal scalar predecessor bootstrap*, frozen in `Research/certified_zero/reference/scalar_predecessor.tex`, pinned commit. Reproduced in Appendix C.

[7] ProveIt contributors. *From literal-equation induction to full intuitionistic PRA*, frozen in `Research/certified_zero/reference/equality_reflection.tex`, pinned commit. Reproduced in Appendix D.

[8] ProveIt contributors. *Pairing, primitive recursion, and internal numerical composites*, frozen in `Research/certified_zero/reference/primitive_recursion.tex`, pinned commit. Reproduced in Appendix E.

[9] ProveIt contributors. `Lean/ConciseIPRA/NativeZeroSource.lean`, pinned commit. https://github.com/VladimirReshetnikov/ProveIt/blob/1517d9d79ce33f631e0f82960c849427cbba23bf/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA/NativeZeroSource.lean.

[10] ProveIt contributors. `Lean/ConciseIPRA/NativeBiInterpretation.lean`, pinned commit. https://github.com/VladimirReshetnikov/ProveIt/blob/1517d9d79ce33f631e0f82960c849427cbba23bf/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA/NativeBiInterpretation.lean.

[11] ProveIt contributors. `Lean/ConciseIPRA/BiInterpretation.lean`, pinned commit. https://github.com/VladimirReshetnikov/ProveIt/blob/1517d9d79ce33f631e0f82960c849427cbba23bf/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA/BiInterpretation.lean.